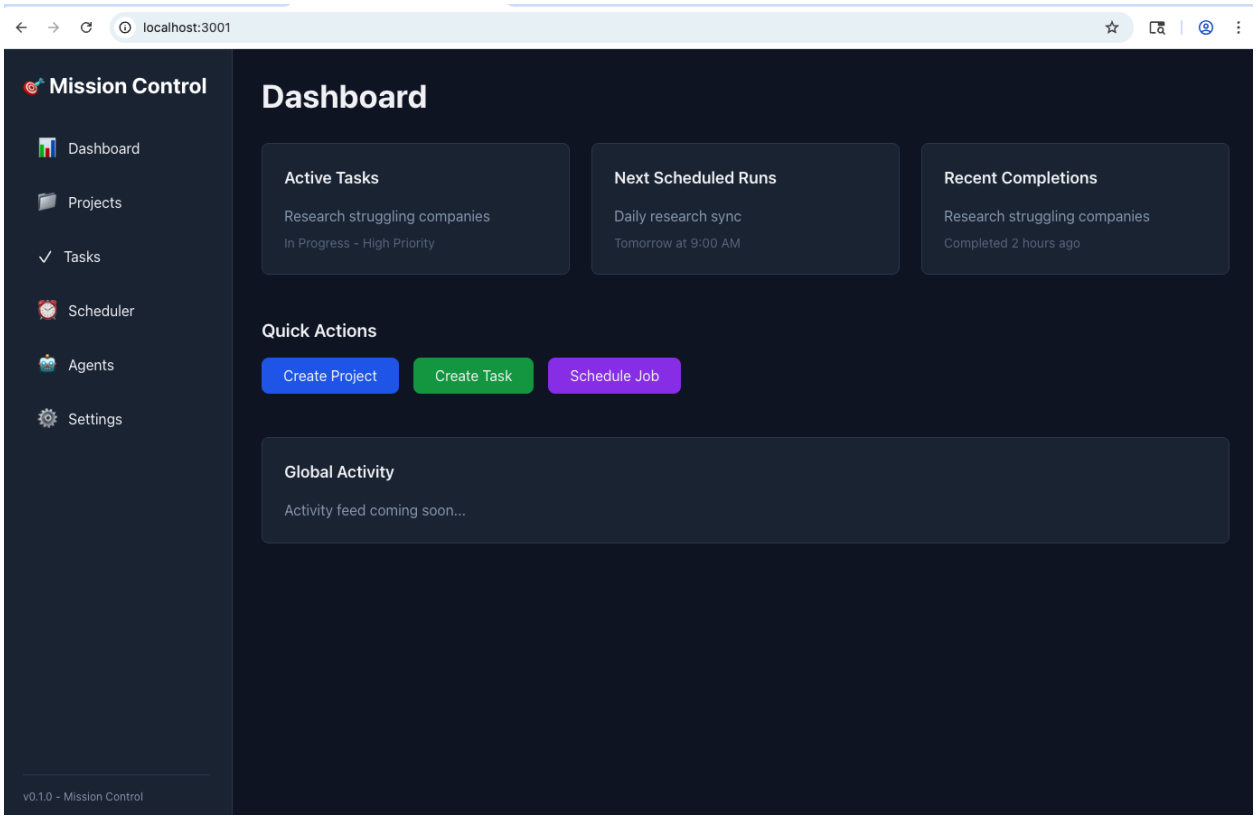
Mission Control

MISSION CONTROL



Overview

OpenClaw Mission Control is your local AI operations dashboard.

It runs at:
http://localhost:3001/
Mission Control does **not** replace the main OpenClaw Web UI.
It is an orchestration layer built for:

- Managing multiple projects
- Structuring execution
- Running agents intentionally
- Scheduling automation
- Tracking outputs and logs
- Operating like a system, not chaos

If OpenClaw is the engine, Mission Control is the command center.

1. Projects

Projects are the foundation of Mission Control.
Every task, agent, schedule, and output is tied to a project.

Creating a Project

When creating a new project, define:

- Project name
- Description
- Project root directory
- Output directory
- Assigned agent(s)
- Optional Git reference

Keep each project isolated.
This prevents cross contamination between builds.

Project Dashboard

Each project includes:

- Active tasks
- Assigned agents
- Scheduled jobs
- Recent runs
- Output artifacts
- Activity timeline

Use this page as your execution overview.

2. Tasks

Tasks are structured work units inside a project.
There are three types:

- Plan
- Build
- Ops

Creating a Task

Each task includes:

- Title
- Description
- Type
- Priority
- Status
- Assigned agent
- Expected deliverables

Tasks move across statuses:
Backlog → Planned → In Progress → Blocked → Done
Use this to manage real execution flow.

Running a Task

Click “Run Task” to trigger execution.
What happens:

1. Mission Control calls the OpenClaw adapter
2. The correct agent is executed
3. Logs stream live
4. Outputs are written to the project output directory
5. Run history is saved

You can monitor:

- Status
- Start time
- End time
- Logs
- Produced artifacts

This is real execution, not simulation.

3. Agents and Sub Agents

Agents are automatically discovered from:
~/.openclaw/openclaw.json
You can:

- View available agents
- Assign agents to projects
- Define a primary agent
- Use sub agents for specialization

Think of this as your AI team structure.
Each project can have a different loadout.

4. Scheduler and Cron Jobs

Mission Control supports recurring automation.

Creating a Scheduled Job

You define:

- Project
- Job type
- Cron expression
- Enabled or disabled state

You can:

- Run immediately
- Enable or disable
- View next run time
- Review run history

The system prevents overlapping runs by default.
Use scheduler jobs for:

- Reporting
- Monitoring
- Outreach
- Maintenance tasks
- Recurring build checks

5. Web Chat (Project Context)

Each project includes a contextual chat interface.
This is not generic prompting.
This is project aware AI.

How Chat Works

1. Select project
2. Select agent
3. Optionally attach:
    - A task
    - Specific files
4. Send message
5. Response streams live

You can:

- Save chat output to task notes
- Create a task from chat
- Export the session to markdown
- Store full chat history

All chat sessions are saved per project.

6. Files and Outputs

Each project includes a file browser for its output directory.
You can:

- Browse directory tree
- Search filenames
- Preview small text files
- Copy file paths

- Open folder in your OS
- Link files to tasks

Artifacts from runs are tracked and organized.
No more digging through random folders.

7. Dashboard Overview

The global Mission Control dashboard shows:

- Active tasks across projects
- Running executions
- Upcoming scheduled jobs
- Recent completions
- Blocked tasks

Use this page to understand system health.

8. Execution Architecture

When a task runs:

1. UI triggers backend API
2. Backend validates project paths
3. OpenClaw CLI is executed via allowlisted command
4. Logs stream via WebSocket
5. Run is persisted in database
6. Output artifacts are indexed

Everything is local.
Everything is tracked.

9. Security Model

Mission Control:

- Binds to 127.0.0.1 by default
- Validates project root paths
- Prevents arbitrary command execution
- Does not store secrets in plain text
- Uses environment variable references where needed

Execution is controlled and bounded.

10. Operational Best Practices

To get the most out of Mission Control:

- Keep projects isolated
- Define clear task deliverables
- Use scheduler for recurring work

- Assign agents intentionally
- Review logs after major runs
- Keep output directories clean and structured

Treat this as infrastructure, not a toy dashboard.

11. What Mission Control Enables

You now have:

- Structured multi project management
- Agent orchestration
- Real execution logging
- Recurring automation
- Contextual AI chat
- Artifact traceability
- System level visibility

This is how you move from prompting to operating.

Option A) From My Agent

OpenClaw Mission Control: Build It Yourself
Step-by-Step Prompts for The Sprint Community

BEFORE YOU START

Prerequisites:
- [ ] Node.js 18+
- [ ] Mac or Linux (Windows via WSL)
- [ ] Code editor (VS Code recommended)
- [ ] Git (optional, but helpful)
- [ ] 30-40 hours of time over 3-4 weeks

Setup (10 minutes):
1. Clone or download the codebase from: [GitHub Link]
2. Run `cd openclaw-control-center`
3. Follow SETUP.md to get the app running locally

Test your setup:
```bash
# Terminal 1 - Backend
cd backend
npm install
npx prisma migrate dev
npm run dev

# Terminal 2 - Frontend
cd frontend
npm install
npm run dev

# Should see:
# Backend: http://localhost:3000
# Frontend: http://localhost:3001
```

PHASE 3: OpenClaw Adapter
Time: 3-4 days | Difficulty: Medium | Core Skill: Child processes & streaming

What You'll Build
A service that:

- Reads your OpenClaw config file
- Discovers available agents
- Executes tasks via child_process
- Streams output to the UI in real-time
- Validates paths for security

Prompt 1: Create the OpenClaw Adapter

Your task:
Create `/backend/src/lib/openclaw.ts` with an `OpenClawAdapter` class.

Requirements:
- Read and parse `~/.openclaw/openclaw.json`
- Extract agent list from config
- Implement `getAgents()` function
- Implement `validatePath()` function (security!)
- Export as singleton

Hint:
- Use Node.js `fs` module to read files
- Use `path.resolve()` for path validation
- Use `os.homedir()` to expand `~` in paths

Verification:
```bash
# In backend directory
npm run type-check  # Should have no errors
```

Code template (fill in the blanks):
```typescript
// /backend/src/lib/openclaw.ts
import fs from 'fs';
import path from 'path';
import os from 'os';

export class OpenClawAdapter {
  private configPath: string;
  private config: any = null;

  constructor(configPath: string = '~/.openclaw/openclaw.json') {
    // TODO: Expand the path (~ → home directory)
    this.configPath = /* YOUR CODE */;
  }
```

```
  private expandPath(filePath: string): string {
    // TODO: Replace ~ with home directory
    if (filePath.startsWith('~')) {
      return path.join(/* YOUR CODE */);
    }
    return filePath;
  }

  async loadConfig(): Promise<any> {
    // TODO: Read and parse the JSON file
    const content = fs.readFileSync(/* YOUR CODE */);
    this.config = JSON.parse(content);
    return this.config;
  }

  async getAgents(): Promise<any[]> {
    if (!this.config) {
      await this.loadConfig();
    }
    // TODO: Return the agents array from config
    return /* YOUR CODE */;
  }

  validatePath(filePath: string, allowlist: string[]): boolean {
    // TODO: Check if filePath is within any allowlist item
    const resolved = path.resolve(filePath);

    for (const allowed of allowlist) {
      const allowedResolved = path.resolve(this.expandPath(allowed));
      // TODO: Check if resolved is within allowedResolved
      if (/* YOUR CODE */) {
        return true;
      }
    }

    return false;
  }
}

export const openclawAdapter = new OpenClawAdapter();
```

Prompt 2: Add Agent Discovery API Endpoint

Your task:
Add a new endpoint to `/backend/src/server.ts` that discovers and stores agents.

Requirements:
- Endpoint: `POST /api/settings/discover-agents`
- Read config using OpenClawAdapter
- Store/update agents in database (Prisma)
- Return list of discovered agents

Hint:
- Use `prisma.agent.upsert()` to create or update agents
- Handle errors gracefully (config file not found, parse errors)

Code template:
```typescript
// Add to /backend/src/server.ts

app.post('/api/settings/discover-agents', async (req, res) => {
  try {
    // TODO: Call openclawAdapter.getAgents()
    const agents = /* YOUR CODE */;

    // TODO: For each agent, upsert it into the database
    for (const agent of agents) {
      await prisma.agent.upsert({
        where: { id: agent.id },
        update: {
          name: agent.name,
          model: agent.model,
          // TODO: Any other fields
        },
        create: {
          id: agent.id,
          name: agent.name || agent.id,
          model: agent.model || 'claude-haiku',
          capabilities: JSON.stringify(agent.capabilities || []),
          // TODO: Any other fields
        }
      });
    }
```

```
    // TODO: Return the updated agents list
    const allAgents = /* YOUR CODE */;
    res.json(allAgents);
  } catch (error) {
    console.error('Agent discovery failed:', error);
    res.status(500).json({ error: 'Failed to discover agents' });
  }
});
```

Test it:
```bash
curl -X POST http://localhost:3000/api/settings/discover-agents

# Should return: [{ id, name, model, capabilities, ... }]
```

Prompt 3: Implement Task Execution

Your task:
Create `/backend/src/services/taskExecutor.ts` that runs a task and streams output.

Requirements:
- Accept task ID, command, and working directory
- Create a TaskRun record in DB
- Spawn child_process with the command
- Capture stdout/stderr
- Broadcast output via WebSocket
- Save final status and logs to DB

Hint:
- Import `spawn` from `child_process`
- Return a Promise that resolves when process exits
- Use WebSocket to broadcast to connected clients

Code template:
```typescript
// /backend/src/services/taskExecutor.ts
import { spawn } from 'child_process';
import { PrismaClient } from '@prisma/client';
import { WebSocketServer } from 'ws';
```

```
export class TaskExecutor {
  constructor(
    private prisma: PrismaClient,
    private wss: WebSocketServer
  ) {}

  async executeTask(
    taskId: string,
    command: string,
    workingDir: string
  ): Promise<string> {
    // TODO: Create TaskRun record
    const taskRun = await this.prisma.taskRun.create({
      data: {
        taskId,
        status: 'Running',
        startTime: new Date(),
        logs: '',
      },
    });

    const streamId = taskRun.id;

    try {
      // TODO: Spawn the process
      const process = spawn(/* YOUR CODE */, {
        cwd: workingDir,
        shell: true,
      });

      let logs = '';

      // TODO: Handle stdout
      process.stdout.on('data', (data) => {
        const chunk = data.toString();
        logs += chunk;

        // TODO: Broadcast to WebSocket subscribers
        /* YOUR CODE */;
      });

      // TODO: Handle stderr
      process.stderr.on('data', (data) => {
        /* YOUR CODE */;
```

```
    });

    // TODO: Wait for process to finish
    const exitCode = await new Promise<number>((resolve) => {
      process.on('exit', (code) => {
        resolve(code || 0);
      });
    });

    // TODO: Update TaskRun with final status
    await this.prisma.taskRun.update({
      where: { id: taskRun.id },
      data: {
        status: exitCode === 0 ? 'Completed' : 'Failed',
        endTime: new Date(),
        exitCode,
        logs,
      },
    });

    return streamId;
  } catch (error) {
    // TODO: Handle errors
    await this.prisma.taskRun.update({
      where: { id: taskRun.id },
      data: {
        status: 'Failed',
        endTime: new Date(),
        logs: String(error),
      },
    });
    throw error;
  }
}

private broadcastTaskStream(streamId: string, event: string, data: string) {
  const message = JSON.stringify({
    type: 'task-stream',
    streamId,
    event,
    data,
    timestamp: new Date().toISOString(),
  });
```

```
    // TODO: Send to all WebSocket clients
    this.wss.clients.forEach((client) => {
      // TODO: Check if client is subscribed to this stream
      if (/* YOUR CODE */ && client.readyState === 1) {
        client.send(message);
      }
    });
  }
}
```

Test it:
```bash
# Create a task first, then:
curl -X POST http://localhost:3000/api/tasks/:id/run

# Should return: { taskRunId: "...", status: "Running" }
```

Prompt 4: Wire Task Execution to API

Your task:
Add `/api/tasks/:id/run` endpoint that calls TaskExecutor.

Requirements:
- Find the task by ID
- Validate it exists
- Call taskExecutor.executeTask()
- Return the stream ID
- Handle errors

Code template:
```typescript
app.post('/api/tasks/:id/run', async (req, res) => {
  try {
    const { id } = req.params;

    // TODO: Find the task
    const task = /* YOUR CODE */;

    if (!task) {
      return res.status(404).json({ error: 'Task not found' });
```

```
    }

    // TODO: Get project details
    const project = /* YOUR CODE */;

    // TODO: Get settings (for allowlist)
    const settings = /* YOUR CODE */;

    // TODO: Create TaskExecutor instance
    const taskExecutor = new TaskExecutor(prisma, wss);

    // TODO: Execute task
    const streamId = await taskExecutor.executeTask(
      task.id,
      'echo "Task executed"', // Stub command for now
      project.projectRootPath
    );

    res.status(202).json({ taskRunId: streamId, status: 'Running' });
  } catch (error) {
    res.status(500).json({ error: String(error) });
  }
});
```

Phase 3 Checkpoint
Verify you can:
- [ ] Discover agents from OpenClaw config
- [ ] Create a task in the UI
- [ ] Click "Run Task" and see it execute
- [ ] Check the database: TaskRun record created
- [ ] See logs streaming via WebSocket (in browser console)

If stuck:
- Check backend logs: `npm run dev` output
- Verify task command works: `echo "test"` in your shell
- Check WebSocket connection: Open browser DevTools → Network → WS

PHASE 4: Scheduler System
Time: 3-4 days | Difficulty: Medium | Core Skill: Cron scheduling

What You'll Build
A scheduler that:
- Loads jobs from database on startup
- Registers jobs with node-cron
- Executes jobs on schedule
- Records execution history
- Updates UI via WebSocket

Prompt 5: Create Scheduler Service

Your task:
Create `/backend/src/services/scheduler.ts` with a `SchedulerService` class.

Requirements:
- Constructor accepts Prisma and WebSocketServer
- `initialize()`: Load all enabled jobs from DB, register them
- `scheduleJob()`: Register a single job with node-cron
- `executeJob()`: Run a job immediately
- `unscheduleJob()`: Stop a scheduled job

Hint:
- Use `import * as cron from 'node-cron'`
- Use `cron.schedule(cronExpression, callback)`
- Use `cron.nextDate()` to calculate next run time

Code template:
```typescript
// /backend/src/services/scheduler.ts
import * as cron from 'node-cron';
import { PrismaClient } from '@prisma/client';
import { WebSocketServer } from 'ws';

export class SchedulerService {
  private scheduledTasks: Map<string, any> = new Map();

  constructor(
    private prisma: PrismaClient,
    private wss: WebSocketServer
  ) {}

  async initialize() {
    // TODO: Load all enabled jobs from DB
    const jobs = /* YOUR CODE */;
```

```
  // TODO: For each job, call scheduleJob()
  for (const job of jobs) {
    /* YOUR CODE */;
  }

  console.log(`Scheduler initialized with ${jobs.length} jobs`);
}

async scheduleJob(job: any) {
  // TODO: Prevent duplicate schedules
  if (this.scheduledTasks.has(job.id)) {
    this.unscheduleJob(job.id);
  }

  try {
    // TODO: Create cron schedule
    const task = cron.schedule(job.cronExpression, async () => {
      await this.executeJob(job.id);
    });

    this.scheduledTasks.set(job.id, task);

    // TODO: Calculate next run time
    const nextRunAt = /* YOUR CODE */;

    // TODO: Update job in DB with nextRunAt
    await this.prisma.schedulerJob.update({
      where: { id: job.id },
      data: { nextRunAt: new Date(nextRunAt.toString()) },
    });
  } catch (error) {
    console.error(`Failed to schedule job ${job.id}:`, error);
  }
}

async unscheduleJob(jobId: string) {
  // TODO: Stop the cron task
  const task = this.scheduledTasks.get(jobId);
  if (task) {
    task.stop();
    this.scheduledTasks.delete(jobId);
  }
}
```

```
async executeJob(jobId: string) {
  // TODO: Find job in DB
  const job = /* YOUR CODE */;

  if (!job) return;

  try {
    // TODO: Create SchedulerJobRun record
    const jobRun = await this.prisma.schedulerJobRun.create({
      data: {
        jobId,
        status: 'Running',
        startTime: new Date(),
      },
    });

    // TODO: If job type is RunTask, execute the task
    if (job.jobType === 'RunTask' && job.targetTaskId) {
      // TODO: Call your TaskExecutor here
      /* YOUR CODE */;
    }

    // TODO: Update job run to completed
    await this.prisma.schedulerJobRun.update({
      where: { id: jobRun.id },
      data: {
        status: 'Completed',
        endTime: new Date(),
        exitCode: 0,
      },
    });

    // TODO: Broadcast update via WebSocket
    this.broadcastSchedulerUpdate(jobId, 'completed');
  } catch (error) {
    console.error(`Job execution failed: ${jobId}`, error);
    // TODO: Handle error
  }
}

private broadcastSchedulerUpdate(jobId: string, status: string) {
  const message = JSON.stringify({
    type: 'scheduler-update',
```

```
      jobId,
      status,
      timestamp: new Date().toISOString(),
    });

    // TODO: Send to all WebSocket clients
    this.wss.clients.forEach((client) => {
      if (client.readyState === 1) {
        client.send(message);
      }
    });
  }
}
```

Prompt 6: Wire Scheduler to Server

Your task:
Update `/backend/src/server.ts` to:
- Instantiate SchedulerService on startup
- Add API endpoints for job management
- Call `initialize()` after server starts

Requirements:
- Add `POST /api/scheduler` (create job)
- Add `GET /api/scheduler` (list jobs)
- Add `POST /api/scheduler/:id/run-now` (execute immediately)
- Add `PUT /api/scheduler/:id/enable` and `/disable`

Code template (endpoints):
```typescript
const schedulerService = new SchedulerService(prisma, wss);

httpServer.listen(PORT, async () => {
  // TODO: Initialize scheduler after server starts
  await schedulerService.initialize();

  console.log(`Server running on port ${PORT}`);
});

app.post('/api/scheduler', async (req, res) => {
  try {
```

```
    const { projectId, name, jobType, cronExpression, timezone, enabled } = req.body;

    // TODO: Validate inputs

    // TODO: Create job in DB
    const job = /* YOUR CODE */;

    // TODO: If enabled, schedule it
    if (enabled) {
      await schedulerService.scheduleJob(job);
    }

    res.status(201).json(job);
  } catch (error) {
    res.status(500).json({ error: String(error) });
  }
});

app.post('/api/scheduler/:id/run-now', async (req, res) => {
  try {
    const { id } = req.params;

    // TODO: Execute the job immediately
    await schedulerService.executeJob(id);

    res.json({ status: 'running' });
  } catch (error) {
    res.status(500).json({ error: String(error) });
  }
});

app.put('/api/scheduler/:id/enable', async (req, res) => {
  try {
    const { id } = req.params;

    // TODO: Find job
    const job = /* YOUR CODE */;

    // TODO: Update enabled = true
    const updated = /* YOUR CODE */;

    // TODO: Schedule it
    await schedulerService.scheduleJob(updated);
```

```
    res.json(updated);
  } catch (error) {
    res.status(500).json({ error: String(error) });
  }
});
```

Phase 4 Checkpoint
Verify you can:
- [ ] Create a scheduler job (POST /api/scheduler)
- [ ] List jobs (GET /api/scheduler)
- [ ] Click "Run Now" and see it execute
- [ ] Enable/disable a job
- [ ] Check database: SchedulerJobRun created with correct status

---

PHASE 5 & 6: Quick Build (Follow These Patterns)

Phase 5: Chat System (2-3 days)
What to build:
- `POST /api/projects/:id/chat` - Send message
- `GET /api/projects/:id/chat` - Get history
- WebSocket handler for streaming responses

Key pattern:
```typescript
// Save user message
await prisma.chatMessage.create({
  data: { chatSessionId, role: 'User', content: message }
});

// Simulate agent response (later: integrate real agent)
const response = `Response to: "${message}"`;

// Save agent message
await prisma.chatMessage.create({
  data: { chatSessionId, role: 'Agent', content: response }
});

// Broadcast
wss.clients.forEach(client => {
```

```
  client.send(JSON.stringify({ type: 'chat-message', content: response }));
});
```

Phase 6: Files System (2-3 days)
What to build:
- `GET /api/projects/:id/files` - List directory
- `POST /api/files/open` - Open folder in OS

Key pattern:
```typescript
import fs from 'fs';
import path from 'path';

function buildFileTree(dirPath, depth = 0) {
  const files = fs.readdirSync(dirPath);
  return files.map(file => ({
    name: file,
    path: path.join(dirPath, file),
    type: fs.statSync(path.join(dirPath, file)).isDirectory() ? 'dir' : 'file',
    children: /* recursively list subdirs */
  }));
}
```

PHASE 7: UI Wiring (Done in Frontend)

Setup State Management

Install Zustand:
```bash
cd frontend
npm install zustand
```

Create store:
```typescript
// /frontend/src/store/appStore.ts
import { create } from 'zustand';

interface AppState {
  projects: any[];
```

```
  tasks: any[];
  selectedProjectId: string | null;

  setProjects: (projects: any[]) => void;
  setTasks: (tasks: any[]) => void;
}

export const useAppStore = create<AppState>((set) => ({
  projects: [],
  tasks: [],
  selectedProjectId: null,

  setProjects: (projects) => set({ projects }),
  setTasks: (tasks) => set({ tasks }),
}));
```

Update Dashboard Component

Replace "Coming Soon" with real data:

```typescript
// /frontend/src/pages/Dashboard.tsx
import { useEffect } from 'react';
import { useAppStore } from '../store/appStore';

export default function Dashboard() {
  const projects = useAppStore(state => state.projects);
  const setProjects = useAppStore(state => state.setProjects);

  useEffect(() => {
    // Fetch projects
    fetch('/api/projects')
      .then(res => res.json())
      .then(data => setProjects(data));
  }, []);

  return (
    <div className="p-8">
      <h1 className="text-4xl font-bold mb-8">Dashboard</h1>

      {/* Real data instead of hardcoded */}
      <div className="grid grid-cols-3 gap-6">
        {projects.map(project => (
          <div key={project.id} className="bg-gray-800 p-6 rounded-lg">
```

```
            <h3 className="text-xl font-semibold">{project.name}</h3>
            <p className="text-gray-400">{project.description}</p>
          </div>
        ))}
      </div>
    </div>
  );
}
```

TROUBLESHOOTING GUIDE

"Port 3000 already in use"
```bash
lsof -ti:3000 | xargs kill -9
```

"Cannot find module 'prisma'"
```bash
npm install
npx prisma generate
```

"Database locked"
```bash
rm backend/prisma/data.db-*
npx prisma migrate dev --skip-generate
```

"WebSocket connection refused"
Check backend is running: `npm run dev` in backend folder

Task execution doesn't stream logs
Check browser DevTools → Network → WS tab for WebSocket messages

---

NEXT STEPS AFTER COMPLETING ALL PHASES

1. **Deploy to a VPS** (DigitalOcean, Render, Fly.io)
2. **Add authentication** (if sharing with team)
3. **Integrate real OpenClaw agents** (sessions_spawn)

4. **Add notifications** (email on task completion)
5. **Build plugins** (custom integrations)

FINAL CHECKLIST

When you're done, you should be able to:
- [ ] Create projects from the UI
- [ ] Create tasks and drag them across a kanban board
- [ ] Click "Run Task" and see logs stream in real-time
- [ ] Create scheduler jobs and see them execute
- [ ] Chat with agents and save conversations
- [ ] Browse project output files
- [ ] All data persists across app restarts
- [ ] No "Coming Soon" text anywhere

If all boxes checked: **You've built Mission Control!**

Questions? Join The Sprint or office hours.

Ready to build?

Option B) My Prompts

Phase 1:

You are my senior full stack engineer and product designer.

New Project: OpenClaw Web UI Control Center (Local Mission Control)
Goal: Build a polished, local hosted “mission control” dashboard that helps me manage all my OpenClaw projects, tasks, sub agents, automations, and output folders in one place.

Important context
- This is NOT replacing the existing OpenClaw Web UI. This is an optional second dashboard focused on operating multiple projects at once.
- I run many projects in parallel and I need strict separation by project.
- This must be hosted locally (LAN only by default). It can run on Mac or Linux, and optionally on a VPS later, but build for local first.
- It must integrate with my OpenClaw configuration and filesystem outputs. It should link to output folders and open files quickly.

Primary user
- Me (operator) and optionally a small internal team
- I need quick visibility, fast triage, and clear next actions across projects.

High level outcomes
1) One dashboard to view all projects and their status at a glance
2) Create and manage projects, tasks, and sub agents with clear workflows
3) Track active tasks, queued tasks, and completed tasks with logs and artifacts
4) Built in web chat that can talk to an OpenClaw agent in the context of a selected project
5) A cron and scheduler area for recurring jobs (view, create, edit, run now, enable/disable)
6) Strong file and output management: link to folders, browse artifacts, search, and open in OS file explorer
7) Clean UI, fast, reliable, with an operational feel (mission control)

Non goals
- Do not attempt to rebuild OpenClaw’s entire UI or add features that already exist there.
- Do not add cloud dependencies. Local first. Offline friendly.
- No heavy enterprise auth. Keep it simple and secure for local use.

Tech requirements (choose a modern but simple stack)
- Frontend: React + TypeScript + Tailwind
- Backend: Node.js (Express or Fastify) + TypeScript
- DB: SQLite (via Prisma or Drizzle). Local file stored under a project data directory.
- Real time: WebSocket for task status updates and chat streaming.
- Packaging: Docker Compose optional, but must also run via npm scripts locally.

- OS integration: Provide “Open Folder” and “Reveal in Finder” style actions (Mac) and equivalent for Linux. If direct opening is not possible from browser, implement a backend endpoint that triggers OS open via shell command with strong path validation.

Integration requirements
- OpenClaw config file lives at: ~/.openclaw/openclaw.json
- The Control Center must be able to read that config (read only by default).
- It should detect agents defined in config and show them as available “sub agents”.
- It should support a per project mapping to:
  - which agent(s) are used
  - working directory root
  - output folder root
  - environment variables or secrets references (do not store secrets in plain text)
- It must show running tasks and completed tasks, and point to their output artifacts on disk.

Core features (must build)
A) Projects
- Create new project wizard
  - Name, description, tags
  - Project root directory path
  - Output directory path
  - Default agent or agent group (from openclaw.json)
  - Optional Git repo link (just metadata)
  - Operations cadence (daily, weekly) metadata
- Project detail page
  - Overview: status, last run, open tasks, next scheduled run
  - Tasks board: Backlog, Planned, In Progress, Blocked, Done
  - Agents: assigned agents, agent notes, quick actions
  - Files: output folders, artifacts, recent files
  - Activity timeline: task events, scheduler runs, chat summaries

B) Tasks
- Task types
  1. Plan task (research, PRD, roadmap)
  2. Build task (feature build, bug fix, implementation)
  3. Ops task (monitoring, outreach, reporting, recurring maintenance)
- Task fields
  - Title, description, type, priority, status, due date
  - Project, assignee agent, dependencies
  - Links: docs, URLs
  - Output folder pointer and expected deliverables checklist
- Task execution
  - “Run task” button that triggers an OpenClaw run via a backend command integration (see integration section below)

  - Store logs, start and end timestamps, exit status
  - Capture produced artifacts list (scan output folder before and after run, then diff)

C) Agents and sub agents
- Agents index page
  - Show all agents discovered from ~/.openclaw/openclaw.json
  - Show which projects use which agents
  - Notes and capabilities fields (editable in Control Center DB, not written back to openclaw.json unless explicitly enabled later)
- Per project “agent loadout”
  - Define which agent is primary and which are sub agents
  - Quick “chat as agent” from the project context

D) Scheduler (cron job setup area)
- Scheduler page
  - List jobs: name, project, schedule (cron expression), enabled, last run, next run
  - Create job wizard
    - Select project
    - Select job type: run task, run agent prompt, run maintenance script
    - Cron expression builder plus raw input
    - “Run now” button
  - Execution log history for each job
- Implementation detail
  - Use a Node scheduler (like node-cron) that is always on while backend is running
  - Save schedules to SQLite, reload on startup
  - Add safe guards to prevent overlapping runs unless explicitly allowed
  - Provide timezone handling and show timezone clearly in UI

E) Web chat (per project)
- Chat UI inside each project
  - Select agent
  - Chat messages stream in real time
  - Include context badges: project, selected task (optional), selected files (optional)
  - Provide “Send output to task notes” and “Create task from chat”
- Minimal chat memory
  - Store chat sessions per project in SQLite
  - Provide searchable chat history
  - Allow exporting a chat session to a markdown file in the project output directory

F) Files and outputs
- Files tab per project
  - Show output directory tree
  - Search filenames and basic text search for small files
  - “Open folder” action

  - “Copy path” action
  - “Attach file to task” and “Link file to task”
- Artifact indexing
  - Background indexing process (local) that updates recent files list and simple metadata
  - Must not read secrets or scan outside configured directories

G) Global Mission Control dashboard
- Home screen widgets
  - Active tasks across all projects
  - Next scheduled runs
  - Recent task completions
  - Projects with blockers
  - Quick create: project, task, scheduler job
- A command palette (Ctrl K)
  - Jump to project
  - Create task
  - Run task
  - Open output folder

Integration details (how to run OpenClaw)
- Implement an adapter layer with 2 modes
  Mode 1: Stub mode (for early development)
  - Running a task simulates progress and writes a fake log

  Mode 2: Real OpenClaw integration
  - Provide a backend setting where I specify the OpenClaw CLI command and working directory
  - The system runs a child process, streams stdout and stderr to the UI, saves logs to disk and DB
  - Store per project run configuration: cwd, env var references, output folder
- Safety
  - Validate all paths against allowlisted project roots
  - Do not allow arbitrary shell commands from the UI
  - Only allow running predefined templates and known OpenClaw commands
  - Provide an “admin settings” page to manage allowlists

Security and privacy
- Local only by default
  - Bind backend to 127.0.0.1
  - Add an optional basic auth toggle for LAN
- Secrets
  - Do not store API keys in SQLite in plain text
  - Provide a way to reference environment variables
  - Provide a settings page that shows which env vars are required per project

UX and design requirements
- Polished mission control style UI
- Fast navigation, minimal clutter
- Clear project separation
- Consistent components, good spacing, modern look
- Dark mode optional but preferred

Deliverables required from you (agent)
1) A product spec
  - User stories
  - Screens list
  - Navigation map
  - Data model
  - Permissions model (even if simple)
  - Edge cases

2) A technical design doc
  - Architecture diagram (described in text)
  - API routes list
  - DB schema (tables and key fields)
  - WebSocket events
  - Scheduler design
  - File indexing approach
  - OpenClaw integration adapter approach
  - Security considerations and path validation strategy

3) Implementation plan
  - Milestones (at least 5)
  - Each milestone includes specific tickets
  - Identify what can be stubbed first
  - Identify what needs real integration later

4) A working MVP implementation
  - Repo structure
  - Backend server with SQLite
  - Frontend dashboard with routing
  - Projects CRUD
  - Tasks board CRUD
  - Scheduler CRUD with node-cron
  - Chat UI with stub streaming
  - File browser for output directories
  - WebSocket live updates for task runs
  - Basic settings page for OpenClaw config path and allowlists

Acceptance criteria
- I can create a project in under 60 seconds
- I can create tasks and move them across statuses with drag and drop
- I can click “Run task” and see streaming logs and status updates
- I can create a scheduled job and it runs at the next schedule
- I can open the output folder for a project from the UI
- I can chat with an agent in a project context and save the chat to a task note
- The app runs locally with a single command and persists data across restarts

Constraints and quality bar
- Prefer simple, reliable choices over flashy ones
- Strong input validation throughout
- Handle errors with helpful messages
- No em dashes anywhere in UI text, logs, or docs
- Provide meaningful sample data for demo

Now do the following, in order
Step 1: Ask any critical questions only if truly blocking. Otherwise assume sensible defaults.
Step 2: Produce the product spec and technical design doc.
Step 3: Produce the implementation plan with milestones and tickets.
Step 4: Generate the initial codebase with clear instructions to run locally.
Step 5: List immediate next improvements after MVP.

Output format
- Use clear headings
- Use code blocks for schema, routes, and commands
- Provide copy paste run commands
- Keep docs and code consistent

Start now.

Phase 2: Wiring back end (100 hours to build this)

You are my senior full stack engineer and systems architect.

Project: OpenClaw Mission Control
Current Status: The app is already live and running at http://localhost:3001/

Important:
- This is NOT a greenfield build.
- Do NOT rebuild the project from scratch.

- We are refactoring, wiring, and upgrading the existing live app.
- Many UI sections currently show “Coming Soon”.
- Buttons exist but are not wired to backend logic.
- We must turn this into a fully functional Mission Control.

Primary Goal
Modernize the UI to feel futuristic and then wire every function end to end so that:
- Projects
- Tasks
- Agents
- Task runs
- Scheduler / cron jobs
- Logs
- Files
- Web chat
- Streaming updates

…are all real and integrated with OpenClaw.

This must operate locally and integrate with:
~/.openclaw/openclaw.json

==========
PHASE 1: CODEBASE AUDIT
==========

Before changing anything:

1. Inspect the running app architecture:
   - Frontend framework
   - Backend framework
   - DB layer
   - WebSocket implementation
   - Routing structure
   - Current schema

2. Identify:
   - Which UI components are placeholders
   - Which buttons lack onClick handlers
   - Which handlers exist but lack backend endpoints
   - Which endpoints exist but do not persist
   - Which backend services are stubs
   - Where “Coming Soon” is hardcoded

3. Produce:
   - A wiring gap report
   - A list mapping UI actions → missing backend logic
   - A list mapping backend endpoints → missing DB writes
   - A list mapping OpenClaw integration points → not implemented

Do NOT skip this audit.

==========
PHASE 2: WIRING PLAN
==========

Create a full wiring map:

For every major feature, define:

UI Action
→ API Route
→ DB Mutation
→ OpenClaw Adapter Action
→ WebSocket Event
→ UI State Update

Cover:

1) Projects CRUD
2) Tasks CRUD + drag and drop status changes
3) Run Task button
4) Agent discovery from ~/.openclaw/openclaw.json
5) Assign agents to project
6) Scheduler create / enable / disable / run now
7) Task run log streaming
8) Web chat streaming
9) File browser + open folder action

Nothing may remain unconnected.

==========
PHASE 3: OPENCLAW ADAPTER (REAL)
==========

Implement a backend OpenClaw adapter layer.

Requirements:

- Read ~/.openclaw/openclaw.json
- Discover available agents
- Allow per project configuration:
  cwd
  output directory
  selected agent
  environment references

- Execute OpenClaw via child_process
- Stream stdout/stderr via WebSocket
- Persist logs to disk
- Persist run records to DB
- Support cancellation
- Strict path validation
- No arbitrary command execution

All execution must:
- Be allowlisted
- Validate project root boundaries
- Bind backend to 127.0.0.1

==========
PHASE 4: SCHEDULER SYSTEM
==========

Implement real cron functionality.

- node-cron or equivalent
- Jobs stored in SQLite
- Reload on server restart
- Prevent overlapping runs per project by default
- Record:
  job started
  job completed
  job failed
- WebSocket updates for scheduler runs

UI must allow:
- Create job
- Edit cron expression
- Enable/disable
- Run now
- View history

- See next run time

==========
PHASE 5: WEB CHAT (REAL)
==========

Implement per-project chat.

- Select agent
- Stream responses live
- Store chat sessions
- Allow:
  Save to task notes
  Create task from chat
  Export to markdown in project output folder

If OpenClaw chat endpoint exists, use it.
If not, create a ChatAdapter interface that uses OpenClaw CLI mode.

No “Coming Soon”.
Chat must function.

==========
PHASE 6: FILES SYSTEM
==========

Implement per-project file browser.

- Tree view
- Filename search
- Preview small text files
- Copy path
- Open folder via backend command
- Link file to task

Must validate all paths.

==========
PHASE 7: FUTURISTIC UI MODERNIZATION
==========

Upgrade visual design without breaking wiring.

Requirements:

- Dark mode default
- Subtle glow accents
- Glass panels
- Clean typography scale
- Animated state transitions
- Real time indicators
- Command palette (Ctrl K) wired
- No clutter
- No em dashes anywhere

Do not introduce performance issues.

UI must feel like:
“AI Operations Command Center”

==========
DATABASE REQUIREMENTS
==========

Ensure schema supports:

Projects
Tasks
TaskRuns
Agents
ProjectAgents
SchedulerJobs
SchedulerRuns
ChatSessions
ChatMessages
Settings
Artifacts

Provide final schema.

==========
ACCEPTANCE CRITERIA
==========

Every button works.
Every section is functional.
No placeholder text remains.
Task runs stream logs live.

Scheduler executes and logs.
Agents load from openclaw.json.
Chat streams and saves.
Files browser works.
App persists across restart.
Dashboard shows real data.

==========
OUTPUT FORMAT
==========

1) Audit report
2) Wiring map
3) Updated architecture
4) Final DB schema
5) Milestone plan (at least 6)
6) Implementation tasks
7) Smoke test checklist
8) UI modernization spec

Do not rebuild from scratch.
Upgrade and wire the existing live system.

Start with the audit. Respond back with an estimate of cost and token usage.